

Reliable Machine-Learning Pipelines

Bind validation, preprocessing, model selection, and serialization into a reproducible workflow.



WHY THIS MATTERS

Start from what you already know

Data quality, training/test boundaries, regression, classification, and evaluation metrics

- 01 Explain imputation and encoding in plain language.
- 02 Use ColumnTransformer and Pipeline in a small worked example.
- 03 Complete the guided lab and verify experiment artifacts.

CORE CONCEPT

The vocabulary of this topic

Bind validation, preprocessing, model selection, and serialization into a reproducible workflow.

imputation and encoding: Imputation replaces declared missing values; encoding converts categories into numerical model inputs.

ColumnTransformer and Pipeline: ColumnTransformer applies column-specific operations; Pipeline binds preprocessing and estimation in order.

cross-validation: Cross-validation repeats training and validation across folds to estimate variation in performance.

experiment artifacts: Experiment artifacts are versioned models, preprocessors, metrics, reports, and configurations produced by a run.

FOUR CONNECTED IDEAS

How the concept works

01



**imputation and
encoding**



02

**ColumnTransformer
and Pipeline**



03

cross-validation



04

**experiment
artifacts**

Fit learns preprocessing and model parameters from each training fold; transform and predict reuse those fitted values.

CORE CONCEPT

Read every part of the mechanism

Fit learns preprocessing and model parameters from each training fold; transform and predict reuse those fitted values.

imputation: A declared replacement for missing values.

pipeline: An ordered fitted workflow.

fold: One cross-validation partition.

WORKED EXAMPLE

Worked example by hand

Show why scaling all rows before the split leaks the test mean, then place scaling inside a Pipeline.

01

Known values

02

Apply the mechanism

03

Interpret the result

RUN IT AND INSPECT THE RESULT

Map the concept to Python

```
model = make_pipeline(StandardScaler(), Ridge())  
scores = cross_val_score(model, X, y, cv=5, scoring="neg_root_mean_squared_error")  
print(f"folds={len(scores)}")
```

OUTPUT

folds=5

Each line corresponds to a concept already explained.

QUALITY GATE

Build the complete mental model

- ☒ Fit imputation, scaling, and encoding inside each training fold to prevent leakage.

- ☒ Cross-validation reports variation across folds, not one deceptively precise score.

- ☒ Select hyperparameters using validation evidence and evaluate once on the untouched test set.

- ☒ Persist preprocessing, model, schema, versions, parameters, metrics, and data identity together.

FOUR CONNECTED IDEAS

Notebook readiness map

01

**imputation and
encoding**

02

**ColumnTransformer
and Pipeline**

03

cross-validation

04

**experiment
artifacts**

Use the concepts from this lesson to read and complete
<https://github.com/curiously/Al-Bootcamp/blob/master/03.model-development.ipynb>.

RISK REVIEW

Common beginner mistakes

01 **Fitting preprocessing before cross-validation.**

02 **Selecting a model on the final test set.**

GUIDED LAB

Build a mixed-column Pipeline, compare candidates with cross-validation, refit the selected model, and serialize it.

01

02

03

04

Predict

Run

Inspect

Explain

SESSION SYNTHESIS

Keep the system, not isolated definitions.

01 Explain imputation and encoding in plain language.

02 Use ColumnTransformer and Pipeline in a small worked example.

03 Complete the guided lab and verify experiment artifacts.

NEXT EVIDENCE

A reproducible training pipeline with schema checks, tracked parameters, validation metrics, and a loadable model.